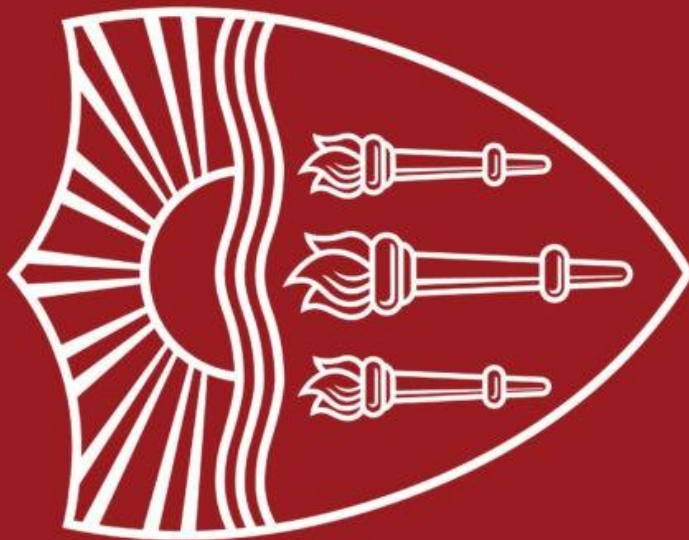




Lecture 3: n-gram LMs and Smoothing + Logistic Regression

Instructor: Xiang Ren
USC CSCI 544 Applied NLP
Fall 2026

Lecture Outline

- Announcements + Recap
 - n-gram Language Models
 - Zeros!
- Smoothing
- Basics of Supervised Machine Learning
 - I. Data: Preprocessing and Feature Extraction
 - II. Model:
 - I. Logistic Regression
 - III. Loss
 - IV. Optimization Algorithm
 - V. Inference

Announcements

Logistics and Announcements

- Always check the website and pay attention to Brightspace Announcement

Class Project

- There are some posts looking for teams, check those posts if you need !
- Mandatory 6 members/team
- Can be mixed among DEN & in-person students. But each one needs to contribute to the project!

Office Hour (update!)

- Friday 8:45-9:30am at GCS SB5

Course video recordings are available on Brightspace

TA update!



TA: Sayan Gosh



TA: Huijuan Wang



TA: Shicheng Wen



TA: Xu Wang



TA: Hirona Arai

Guest Lectures

10	Oct 27	Large Language Models - Evaluation (Guest lecture by Brihi Joshi)	J&M, Chap 10; Chap 12	Project Midterm Report Due
10	Oct 29	Large Language Models - In-context Learning, Scaling Law	J&M, Chap 12	—
11	Nov 3	LLMs - Post-Training (SFT, RLHF) (Guest lecture by Sayan Gosh)	J&M, Chap 12	—
11	Nov 5	Guest Lecture on LLMs for mental health and medical diagnoses (by Prof. Ruishan Liu)	—	—
12	Nov 10	Guest Lecture on LLMs (by Prof. Yue Zhao)	—	HW2 Due
12	Nov 12	Guest Lecture on Accountability and Forensics in the Era of Closed Language Models" (by Matthew Finlayson)	—	—
13	Nov 17	Guest Lecture on LLMs (by Prof. Robin Jia)	—	—

Class Project

- Styled like a research or demo project
- Teams of 6 students
 - We expect to see 47 groups, based on current enrollment of 280 students
 - Exception case: **two groups will be 5 students** – let us know if you prefer
- **Project team formation due by Sep 3**

Pay attention to the website update!

https://inklab.usc.edu/cs544_f26/details/project/

Project Team Formation (due Sep 3)

On Brightspace:

- Navigate to **Course Tools → Groups → View Available Groups**.
- Coordinate with your teammates to join the same empty group (showing **0/6**).

Groups Help

[View My Groups](#)

Available Groups

Join an available group from each category listed.

Groups	Description	Members	Actions
Sample Group			
Group 1		0/4	Join Group
Group 2		0/4	Join Group
Group 3		0/4	Join Group

(detailed instructions [here](#))

Class Project - Proposal

Due on Sep 10 → each team submit one on Brightspace
5% of total grade

The project proposal (**1 page, single-space, 10pt font size**) should outline the type of project (research-focused or application-focused), and then answer the following questions clearly in a sentence and/or a few paragraphs each:

- What are you trying to do? Articulate your objectives using absolutely no jargon.
- How is it done today, and what are the limits of current practice?
- What is new in your approach and why do you think it will be successful?
- Who cares?
- If you are successful, what difference will it make?
- What are the risks? What could go wrong, and how will you pivot early on if that happens?
- How much will it cost? That is, what resources will you need in terms of time and computation? Are these reasonable for a semester and what access you have?
- Identify two milestones along the way to your finished project.

Recap

Probabilistic Language Modeling

Goal: compute the probability of a sentence or sequence of words:

$$P(\mathbf{w}) = P(w_1, w_2, \dots, w_n)$$

Related task: probability of an upcoming word: $P(w_n | w_1, w_2, \dots, w_{n-1})$

A model that assigns probabilities to sequences of words is called a language model

Chain Rule

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-1}, \dots, w_1)$$

How to estimate the probability of the next word?

$$P(\text{that} \mid \text{its water is so transparent}) = \frac{\text{Count}(\text{its water is so transparent that})}{\text{Count}(\text{its water is so transparent})}$$

Maximum Likelihood Estimate

Too many possibilities to count! Too few sentences that look like this...

Need to make some simplifying assumptions...

Markov Assumption

$$P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-k+1}, \dots, w_{i-1})$$

k-th order Markov Assumption

In other words, we approximate each component in the product such that it is only conditioned on the previous *k* - 1 elements

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-k+1}, \dots, w_{i-1})$$

n-gram models

Unigram Model

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i)$$

Bigram Model

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i | w_{i-1})$$

k -gram Model

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i | w_{i-k+1}, \dots, w_{i-1})$$

n-gram Models: Limitations

In general this is an insufficient model of language

- “The computer which I had just put into the machine room on the fifth floor crashed.”

At times the dependencies are not even clear!

- “The complex houses married and single soldiers and their families.”
- “The horse raced past the barn fell.”
- “The old man the boat.”

Garden Path Sentences

But we can often get away with n-gram models

Language has long-distance dependencies

Estimating bigram probabilities

Maximum Likelihood Estimate

$$P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

We do everything in log space to handle overflow issues

Special edge case tokens: $\langle s \rangle$ and $\langle /s \rangle$ for beginning of sentence and end of sentence, respectively

For the 9222 sentences in the Berkeley Restaurant Corpus:

Unigram Counts

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Bigram Counts

History

Next Word

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Most n-grams are never seen!

Bigram Probabilities

W_{i-1}

W_i

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

How good is a language model?

A better model of a text

- is one that assigns a higher probability to the word that actually occurs
- returns a higher probability when evaluated on an unseen test set

Perplexity

$$PPL(\mathbf{w}) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} \quad \text{Normalization Factor}$$

$$= \exp\left(-\frac{1}{N} \log P(w_1, \dots, w_N)\right)$$

Negative log likelihood

Bigram Perplexity

$$PPL(\mathbf{w}) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{i-1})\right)$$

Lower the perplexity, better the language model

WSJ Perplexities

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

n-grams do a better and better job of modeling the training corpus as we increase the value of n

How best to evaluate an LM?

- Extrinsic evaluation
 - On an external task (e.g. summarization) that uses an LM
 - More reliable
 - Can be time-consuming; hard to design
 - Which is the best task? How many tasks to try?
- Therefore, we often use intrinsic evaluation: perplexity
 - Bad approximation (less reliable)
 - Unless the test data looks just like the training data
 - Generally only useful in pilot experiments (faster to compute)

Generating from a bigram model

- Choose a random bigram ($\langle s \rangle$, w) according to its probability
- Now choose a random bigram (w , x) according to its probability
- And so on until we choose $\langle /s \rangle$
- Then string the words together

$\langle s \rangle$ I
I want
want to
to eat
eat Chinese
Chinese food
food $\langle /s \rangle$

I want to eat Chinese food

On your own: Sampling from a probability distribution

Shakespearean n-grams

1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.
–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.
–This shall forbid it should be branded, if renown made it empty.

4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;
–It cannot be but so.

The WSJ is no Shakespeare!

1
gram

Months the my and issue of year foreign new exchange's september
were recession exchange new endorsed a acquire to six executives

2
gram

Last December through the way to preserve the Hudson corporation N.
B. E. C. Taylor would seem to complete the major central planners one
point five percent of U. S. E. has already old M. X. corporation of living
on information such as more frequently fishing to keep her

3
gram

They also point to ninety nine point six billion dollars from two hundred
four oh six three percent of the rates of interest stores as Mexico and
Brazil on market conditions

Shakespearean corpus cannot produce WSJ vocabulary and vice versa

Overfitting!

Two Types of Overfitting Issues

$$P(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- At test time:
 - Zero unigram counts
 - Zero bi-gram counts
 - May lead to undefined n-gram probabilities and perplexity
 - To be expected, very common!
- Solutions:
 - <UNK> token
 - Smoothing

$$PPL(\mathbf{w}) = \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}}$$

N-gram models: Zero Counts

- At test time, we may encounter tokens never seen (unigram with 0 frequency)
 - Very severe yet common problem resulting in undefined probabilities
 - Happens because of new terms, words, different dialects, evolving language
 - These are known as OOV for “out of vocabulary”, or <UNK> for unknown tokens

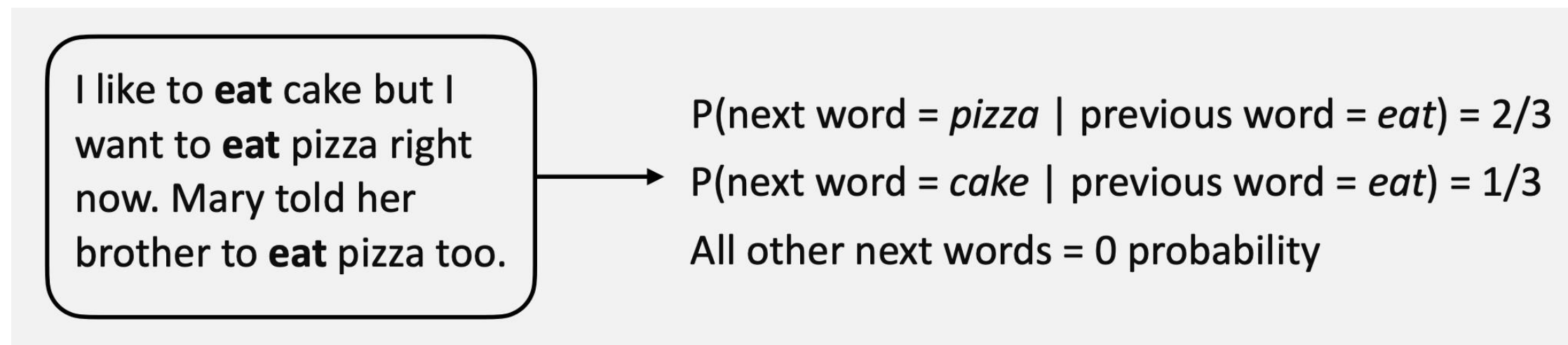
N-gram models: Zero Counts

- Solution: During training (probability estimation), replace all words that occur fewer than n times in the training set by `<UNK>` (where n is some small number) and re-estimate the counts and probabilities.
 - At test time, any OOV token is automatically mapped to `<UNK>`
- **Design:** Open Vocabulary vs. Closed Vocabulary
 - Closed Vocabulary: predetermine the vocabulary (e.g. using a dictionary)
 - Restricted...why?
 - Open Vocabulary: no predetermination but anticipate new tokens

A grayscale photograph of a piece of paper that has been crumpled into a complex, irregular shape. The paper is covered in numerous sharp folds, creases, and wrinkles, creating a highly textured surface. The lighting highlights the ridges and shadows in the folds, giving it a three-dimensional appearance. The word "Smoothing" is written in a clean, white, sans-serif font, centered horizontally and vertically over the crumpled paper. The entire image is set against a solid red background.

Smoothing

Intuition for Smoothing



- Types: I, like, to, eat, cake, but, want, pizza, right, now, ., Mary, told, her, brother, too
 - $|V| = ?$ $|V_{\text{bigrams}}| = ?$
- All other vocabulary tokens getting 0 probability just doesn't seem right. We want to assign some probability to other words
- We want to smooth the distribution from our counts



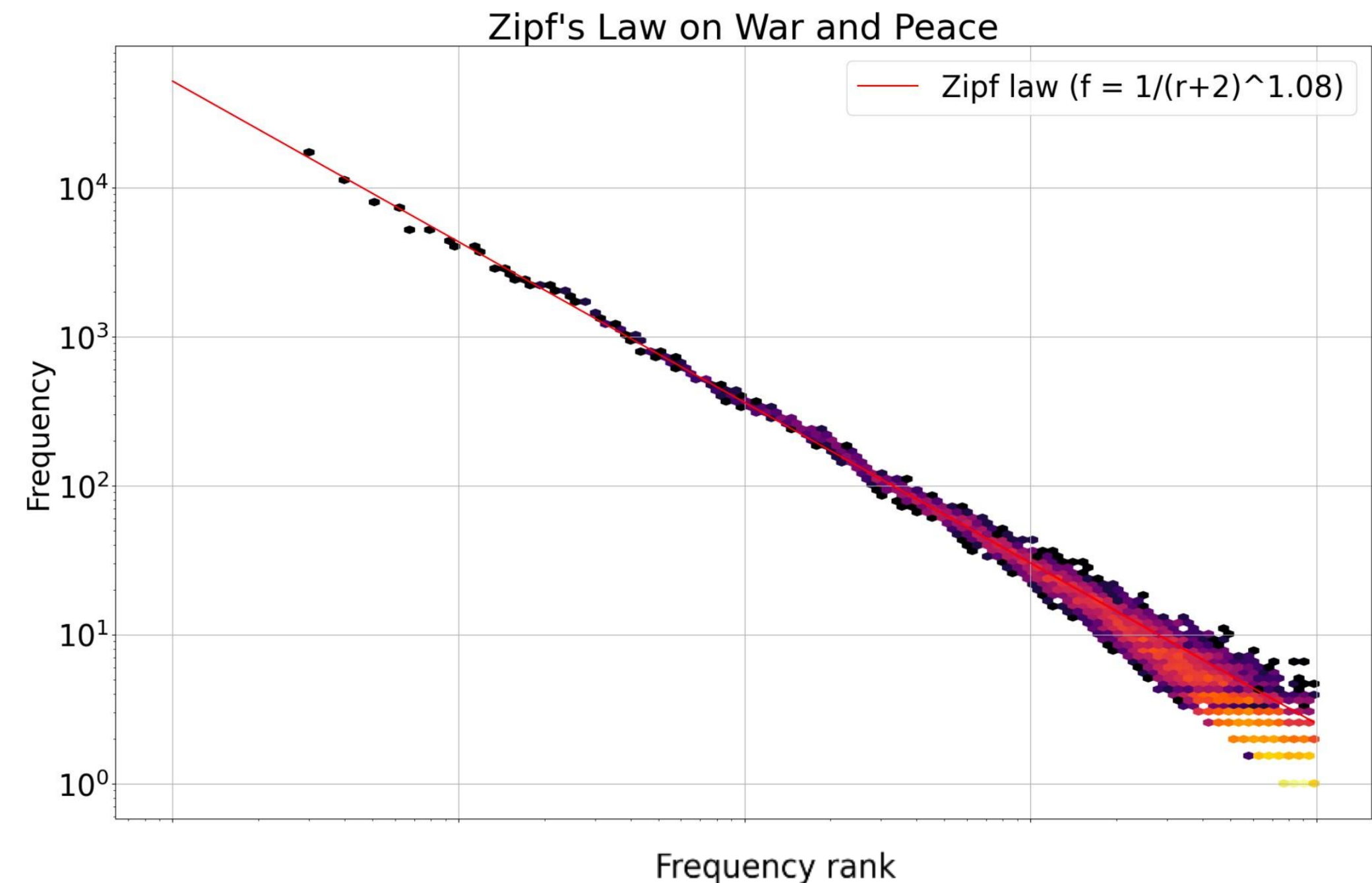
What does a count distribution look like?

Zipf's Law

The distribution over words resembles that of a power law:

- there will be a few words that are very frequent, and a long tail of words that are rare
- $\text{freq}_w(r) \approx r^{-s}$, where s is a constant

NLP algorithms must be especially robust to observations that do not occur or rarely occur in the training data

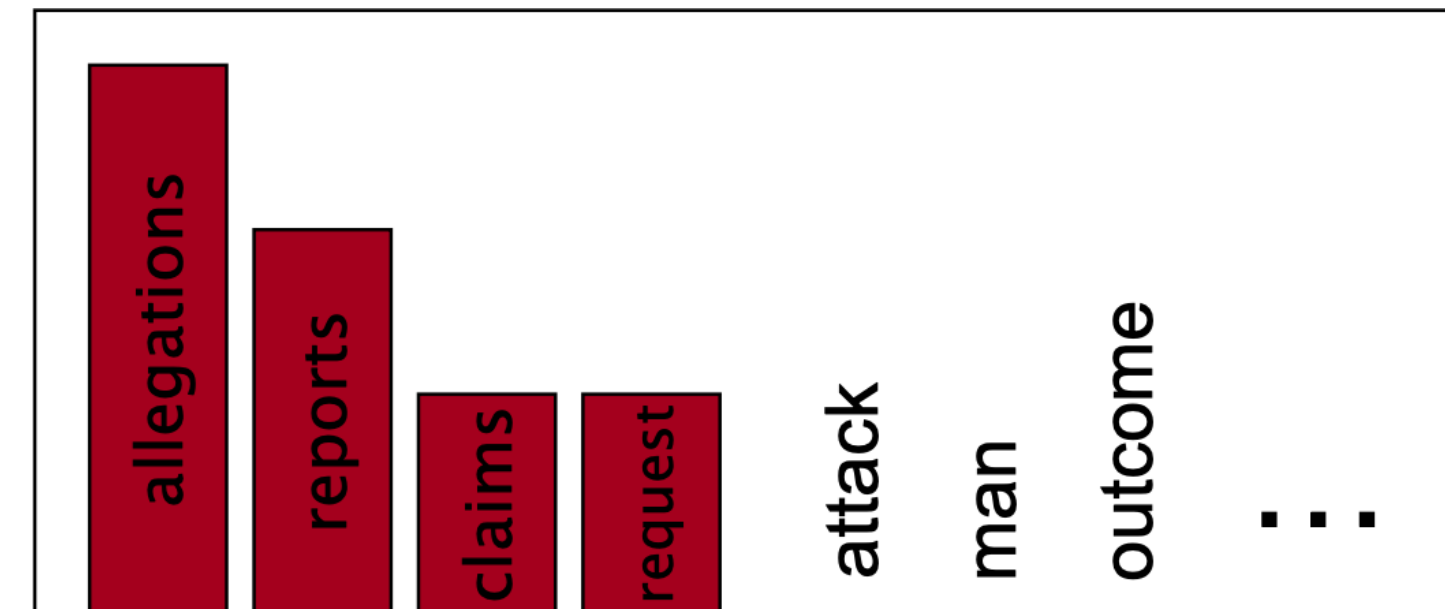


Zipf, G. K. (1949). Human behavior and the principle of least effort.

Smoothing ~ Massaging Probability Masses

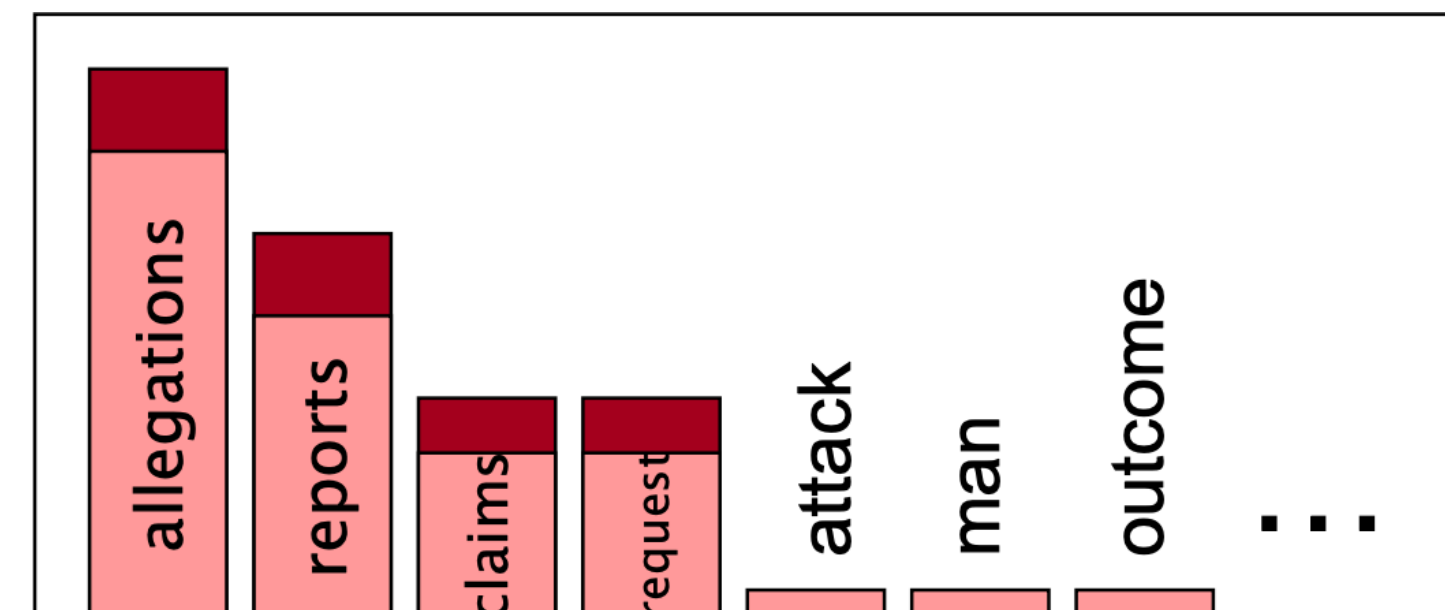
When we have sparse statistics: $Count(w \mid \text{denied the})$

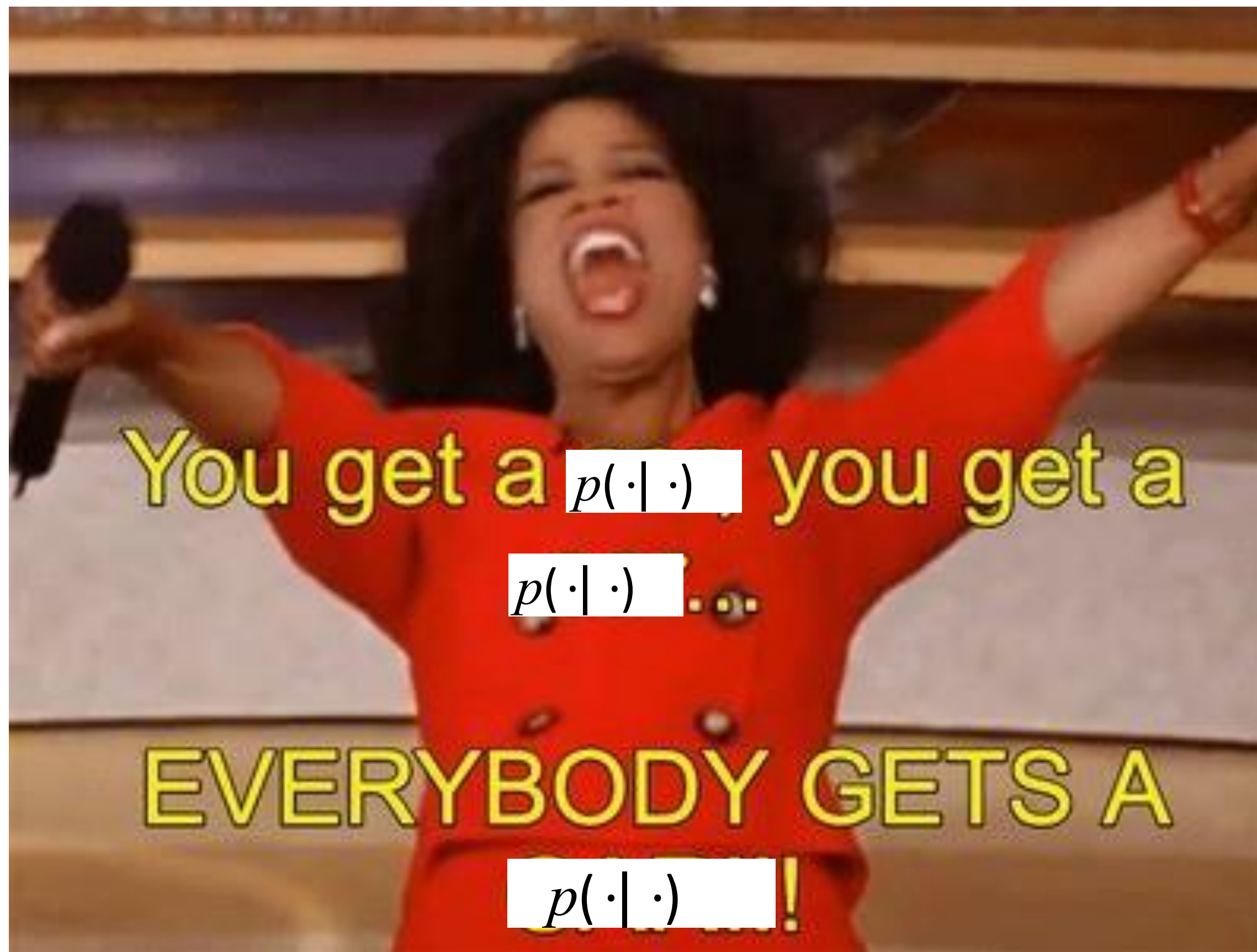
3 allegations
2 reports
1 claims
1 request
7 total



Steal probability mass to generalize better: $Count(w \mid \text{denied the})$

2.5 allegations
1.5 reports
0.5 claims
0.5 request
2 other
7 total





Add-One Estimation

MLE estimate

$$P_{MLE}(w_i) = \frac{c(w_i)}{\sum_w c(w)}$$

Laplace smoothing

1. Pretend we saw each word one more time than we did
2. Just add one to all the counts!
3. All the counts that used to be zero will now have a count of 1...

75 year old method!

Add-1 estimate

$$P_{Add-1}(w_i) = \frac{c(w_i) + 1}{\sum_w (c(w) + 1)} = \frac{c(w_i) + 1}{V + \sum_w c(w)}$$



What happens to our P if we don't increase the denominator?

Add-1 Estimation Bigrams

MLE estimate

$$P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}w_i)}{c(w_{i-1})} = \frac{c(w_{i-1}w_i)}{\sum_w c(w_{i-1}w)}$$

Pretend we saw each bigram one more time than we did

Add-1 estimate

$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}w_i) + 1}{c(w_{i-1}) + V}$$

Recall: BRP Corpus

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i'm looking for a good place to eat breakfast
- when is caffe venezia open during the day

Unigrams

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Bigrams

 w_i
 w_{i-1}

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Laplace-smoothed bigram counts

Original

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Just add one to all the counts!

Smoothed

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Laplace-smoothed bigram probabilities

$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}w_i) + 1}{c(w_{i-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Reconstituting the counts

We can reconstitute the ‘new’ counts c^* for each token

Unigrams:

$$c_i^* = P(w_i) \cdot N$$

$P(w_i)$: probability that the next word is w_i .
 N : number of word tokens we generate

$$= \frac{C(w_i) + 1}{N + V} \cdot N$$

Plug in the model definition of $P(w_i)$
 V : size of vocabulary

$$= (C(w_i) + 1) \cdot \frac{N}{N + V}$$

Rearrange
(to see dependence on N and V)

Bigrams:

$$c^*(w_i | w_{i-1}) = P(w_i | w_{i-1}) \cdot C(w_{i-1})$$

$P(w_{i-1}w_i)$: probability of bigram “ $w_{i-1}w_i$ ”.
 $C(w_{i-1})$: frequency of w_{i-1} (in training data)

$$= \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + V} \cdot C(w_{i-1})$$

Plug in the model definition of $P(w_i | w_{i-1})$

Reconstituted Counts

$$c^*(w_{i-1}w_i) = \frac{[c(w_{i-1}w_i) + 1]c(w_{i-1})}{c(w_{i-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Compare with raw bigram counts

Original, Raw

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Big change
to the counts!

Perhaps 1 is too
much, add a
fraction?

Reconstructed

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Add- k smoothing

k is a
hyperparameter

Add-k Smoothing

Variant of Add-One smoothing:
For any $k > 0$ (typically, $k < 1$)

$$\text{Add K} \quad P(w_i) = \frac{C(w_i) + k}{N + kV}$$

Add-1 Estimation: Last thoughts

So add-1 isn't used for n-grams, being something of a blunt instrument

- One-size-fits-all

Add-1 is used to smooth other NLP models though...

- For text classification (Naïve Bayes)
- In domains where the number of zeros isn't so huge

Interpolation

Perhaps use some pre-existing evidence

- Condition on less in contexts you haven't learned much about

Interpolation

- mix unigram, bigram, trigram probabilities for a trigram LM
- mix n -gram, $(n-1)$ -gram, ... unigram probabilities for an n -gram LM

Interpolation works better than Add-1 / Laplace

Linear Interpolation

We've never seen "*Bob was reading*",
but we might have seen "*__ was reading*",
and we've certainly seen "*__ reading*" (or <UNK>)

$$\begin{aligned}\tilde{P}(w_i | w_{i-1}, w_{i-2}) = & \lambda_3 \cdot \hat{P}(w_i | w_{i-1}, w_{i-2}) \\ & + \lambda_2 \cdot \hat{P}(w_i | w_{i-1}) \\ & + \lambda_1 \cdot \hat{P}(w_i)\end{aligned}\quad \text{for } \lambda_1 + \lambda_2 + \lambda_3 = 1$$

Simple Interpolation

$$\begin{aligned}P_{\text{smoothed}}(w_i = \textit{reading} \mid w_{i-1} = \textit{was}, w_{i-2} = \textit{Bob}) = & \\ \lambda_3 P_{\text{unsmoothed-trigram}}(w_i = \textit{reading} \mid w_{i-1} = \textit{was}, w_{i-2} = \textit{Bob}) & \\ + \lambda_2 P_{\text{unsmoothed-bigram}}(w_i = \textit{reading} \mid w_{i-1} = \textit{was}) & \\ + \lambda_1 P_{\text{unsmoothed-unigram}}(w_i = \textit{reading}) & \end{aligned}$$

Linear Interpolation

Simple Interpolation

$$\begin{aligned}\hat{P}(w_n | w_{n-2}w_{n-1}) &= \lambda_1 P(w_n) \\ &\quad + \lambda_2 P(w_n | w_{n-1}) \\ &\quad + \lambda_3 P(w_n | w_{n-2}w_{n-1})\end{aligned}$$

Different for
every unique
context

Context-Conditional Interpolation

$$\begin{aligned}\hat{P}(w_n | w_{n-2}w_{n-1}) &= \lambda_1(w_{n-2:n-1}) P(w_n) \\ &\quad + \lambda_2(w_{n-2:n-1}) P(w_n | w_{n-1}) \\ &\quad + \lambda_3(w_{n-2:n-1}) P(w_n | w_{n-2}w_{n-1})\end{aligned}$$

How to set the λ s?

Use a **held-out** corpus

Training Data

Held-Out
Data

Test
Data

Choose λ s to maximize the probability of held-out data:

- Fix the n-gram probabilities (on the training data)
- Then search for λ s that give largest probability to held-out set

n-grams Today

Infini-gram: Scaling Unbounded n -gram Language Models to a Trillion Tokens

Jiacheng Liu[♥] Sewon Min[♥]

Luke Zettlemoyer[♥] Yejin Choi^{♥♠} Hannaneh Hajishirzi^{♥♠}

[♥]Paul G. Allen School of Computer Science & Engineering, University of Washington

[♠]Allen Institute for Artificial Intelligence liujc@cs.washington.edu

API Endpoint: If you'd like to issue batch queries to infinigram, you may invoke our API endpoint. Please refer to the [API documentation](#).

Note: The query is **case-sensitive**. Your query will be tokenized with the Llama-2 tokenizer (unless otherwise specified).

The screenshot shows the Infini-gram web interface. On the left is a sidebar titled 'Corpus' with radio buttons for selecting a corpus: OLMo 2 32B Instruct (4.6T tokens), OLMo 2 13B Instruct (4.6T tokens), OLMoE 1B 7B Instruct (4.6T tokens), Dolma-v1.7 (2.6T tokens), RedPajama (1.4T tokens), Pile-train (380B tokens), C4-train (200B tokens), and Pile-val (390M tokens). The main area has tabs for: 1. Count an n-gram, 2. Prob of the last token, 3. Next-token distribution, 4. ∞ -gram prob, 5. ∞ -gram next-token distribution, and 6. Search documents. The '1. Count an n-gram' tab is active. It contains a 'Click to view instructions' link, a 'Query' input field with placeholder text 'Enter a string (an n-gram) here', an 'Advanced options' dropdown, a 'Clear' button, and an orange 'Submit' button. To the right of the query field is a 'Count' button. Below the 'Submit' button are two output fields: 'Latency (milliseconds)' and 'Tokenized'.

- Clever use of smoothing to create n -gram LMs where $n = \infty$, at least in principle
- Trained on several open text corpora: [Dolma](#), [RedPajama](#), [Pile](#), and [C4](#)
 - Same corpora are used to train LLMs
- Several applications: search for n -grams, their counts and their probabilities, use them for generation, etc

Lecture Outline

- Announcements
- Recap
 - n-gram Language Models
 - Zeros!
- Smoothing
- Basics of Supervised Machine Learning
 - I. Data: Preprocessing and Feature Extraction
 - II. Model:
 - I. Logistic Regression
 - III. Loss
 - IV. Optimization Algorithm
 - V. Inference

Basics of Supervised Machine Learning

Ingredients of Supervised Machine Learning

- I. Data as pairs $(x^{(i)}, y^{(i)})$ s.t $i \in \{1 \dots N\}$
 - $x^{(i)}$ usually represented by a feature vector $\mathbf{x}^{(i)} = [x_1, x_2, \dots, x_d]$,
e.g. word embeddings
- II. Model
 - A classification function that computes $\hat{y}$, the estimated class, via $p(y | x)$ e.g. logistic regression, naïve Bayes
- III. Loss

An objective function for learning

 - e.g. cross-entropy loss, L_{CE}
- IV. Optimization
 - An algorithm for optimizing the objective function
e.g. stochastic gradient descent
- V. Inference / Evaluation

Learning Phase

Text Classification Tasks

SENTIMENT ANALYSIS



POSITIVE

"Great service for an affordable price.
We will definitely be booking again."



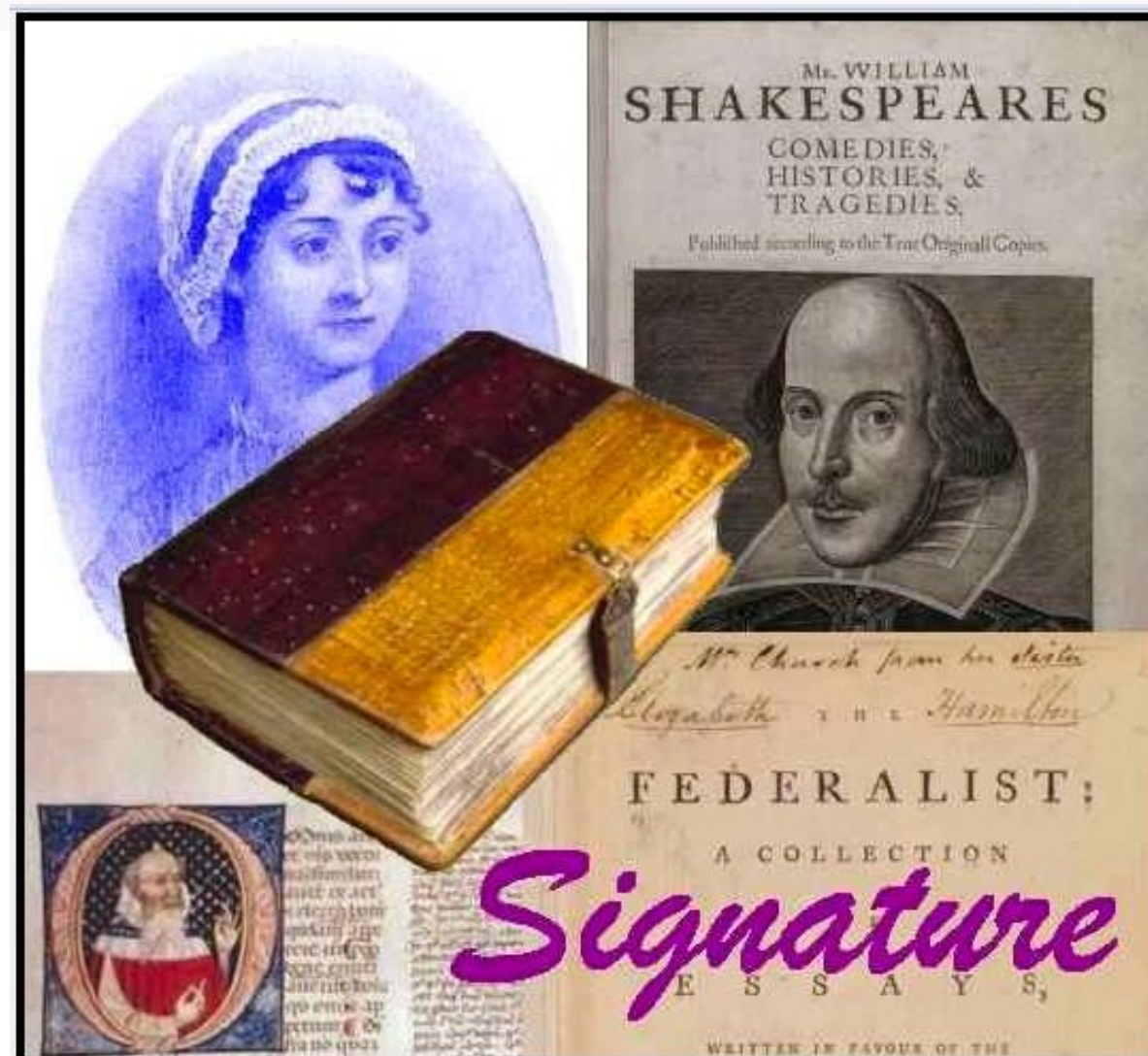
NEUTRAL

"Just booked two nights
at this hotel."



NEGATIVE

"Horrible services. The room
was dirty and unpleasant.
Not worth the money."



★ ID: 133 - Account Alert! (Oct, 2015)



Microsoft account team (outlooo.teeam@outlook.com) [Add to contacts](#) 12:15 AM

To: account-security-nonreply@account.microsoft.com



Dear Outlook user,

You have some blocked incoming mails due to our maintenance problem.

In order to rectify this problem, you are required to follow the below link to verify and use your account normally.

Please click below to unlock your messages, it takes a few seconds.

[Verify Your Account](#)

<http://spapparelsindia.in/Aprons/outlook.com/login.html>

We apologize for any inconvenience and appreciate your understanding.

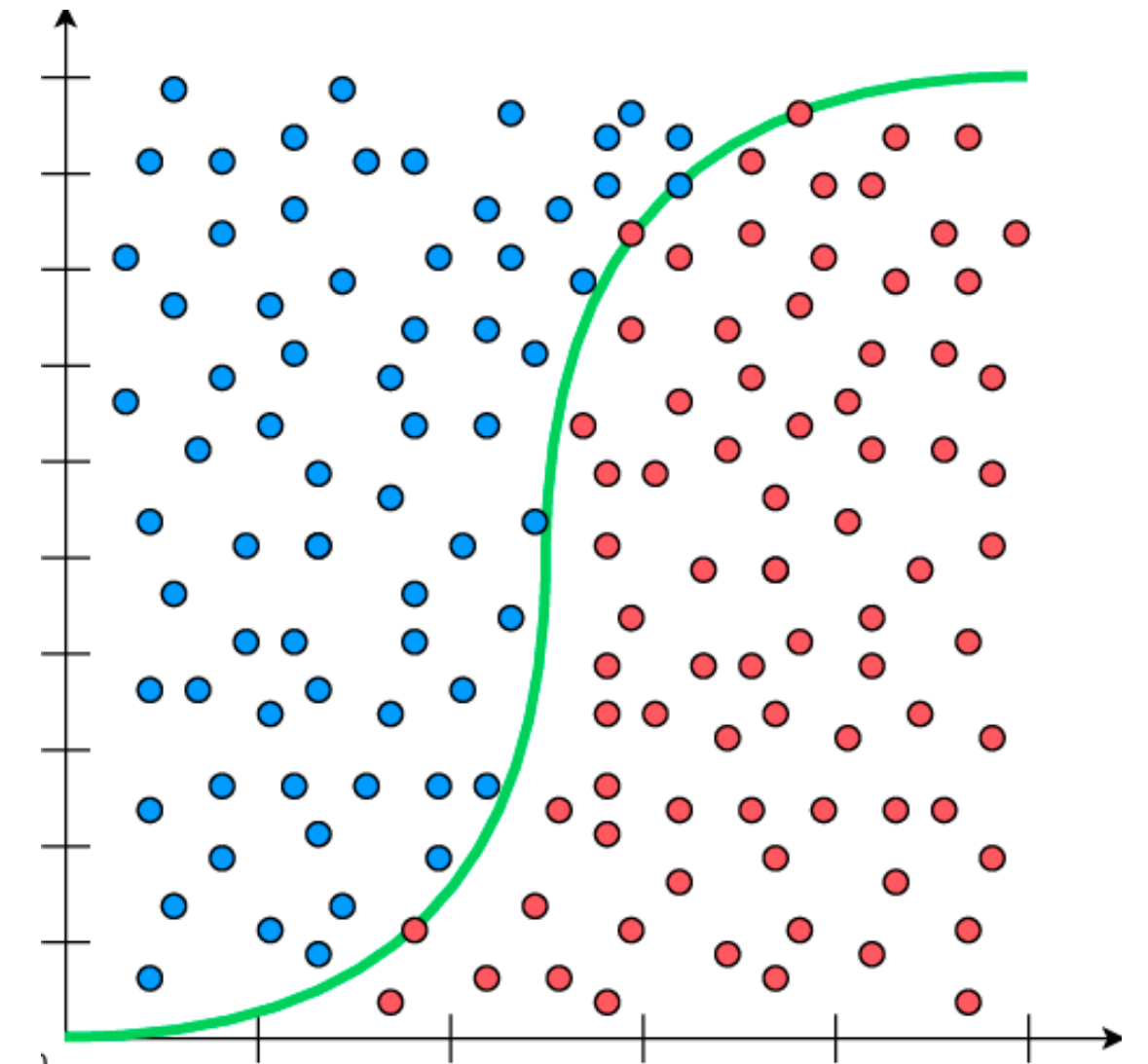
Thanks.

The Microsoft account team™

Not just NLP, classification is a general ML technique often applied across a wide variety of prediction tasks!

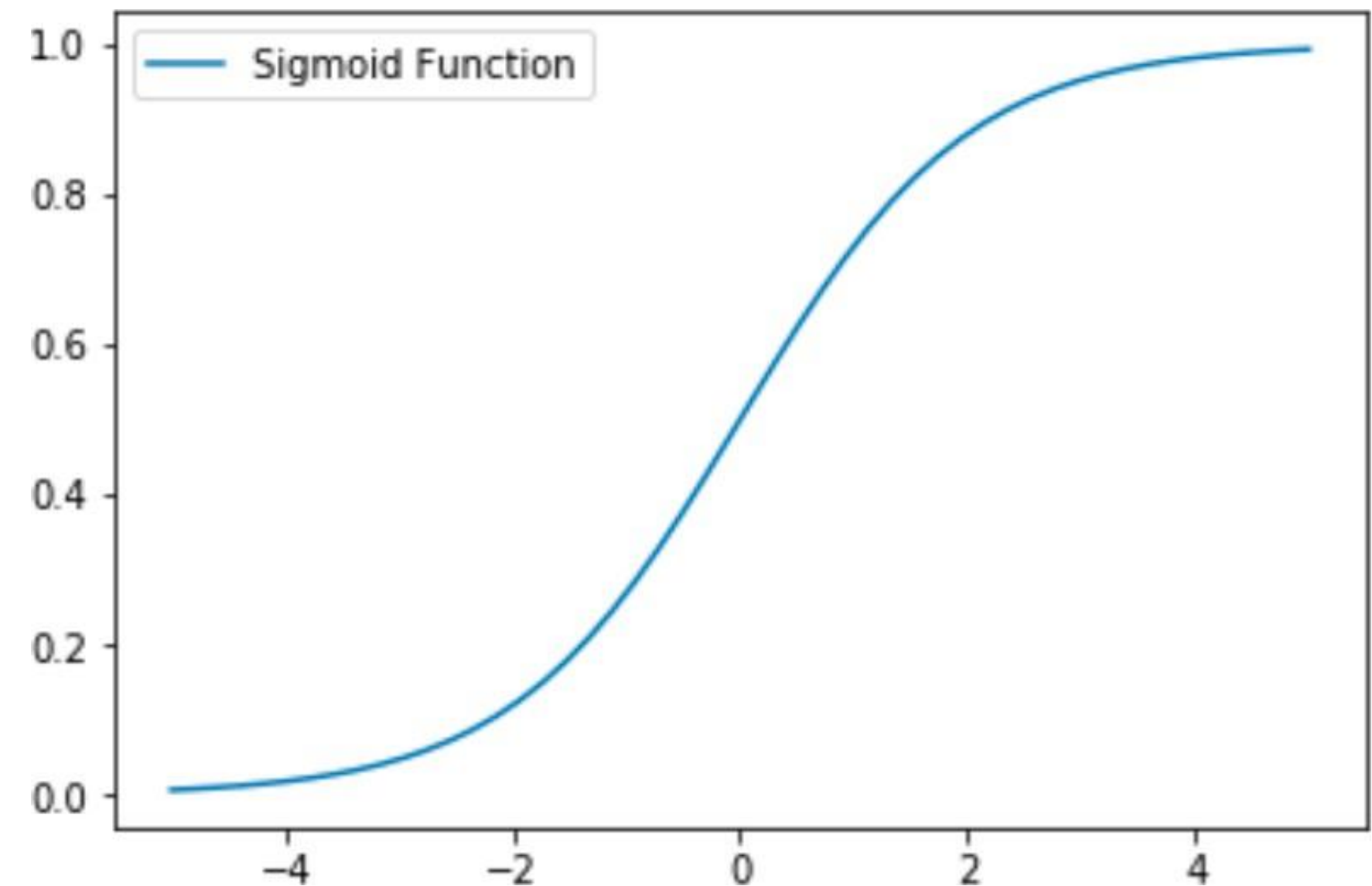
Text Classification Setup

- Input:
 - a document x
 - Each observation $x^{(i)}$ is represented by a feature vector
$$\mathbf{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \dots, x_d^{(i)}]$$
 - a label from a fixed set of classes $C = c_1, c_2, \dots, c_J$
- Output: a predicted class $\hat{y} \in C$
- Setting for Binary Classification: given a series of input / output pairs
 - $(x^{(i)}, y^{(i)})$ where label $y^{(i)} \in C = \{0, 1\}$
- Goal of Binary Classification
 - At test time, for input x^{test} , compute an output: a predicted class $\hat{y}^{test} \in \{0, 1\}$



Example: Logistic Regression

- Important analytic tool in natural and social sciences
- Baseline supervised machine learning tool for classification
- Is also the foundation of neural networks
- Logistic regression is a discriminative classifier
 - Learn a model that can (given the input) distinguish between different classes
- Other classification algorithms: Naïve Bayes, K-Nearest Neighbors, Decision Trees, SVMs



Classification: Single Observation

- Input observation: vector of features, $\mathbf{X} = [x_1, x_2, \dots, x_n]$
- Weights: one per feature: $\mathbf{W} = [w_1, w_2, \dots, w_n]$
 - Sometimes we call the weights $\Theta = [\theta_1, \theta_2, \dots, \theta_n]$
- Output: a predicted class
 - Binary logistic regression $\hat{y} \in \{0, 1\}$
 - Multinomial logistic regression (e.g. 5 classes): $\hat{y} \in \{0, 1, 2, 3, 4\}$

Parametric Model

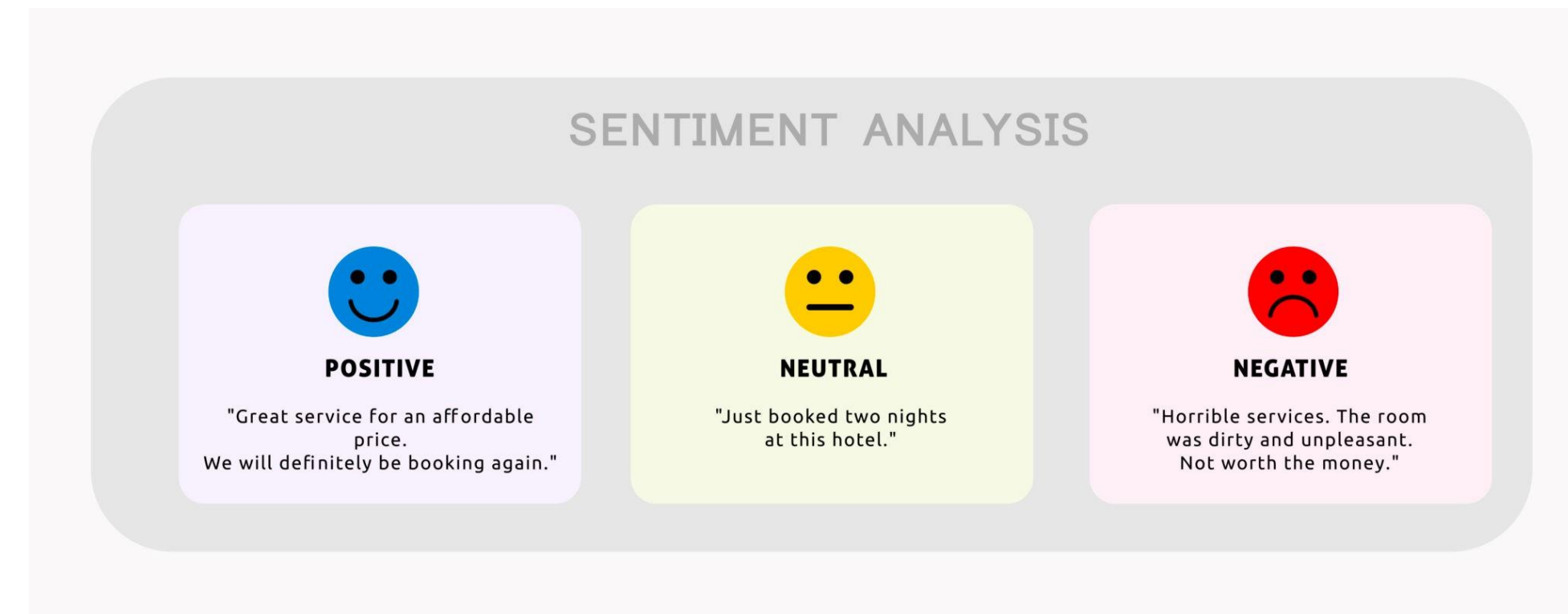
Lecture Outline

- Announcements
- Recap
 - n-gram Language Models
 - Zeros!
- Smoothing
- Basics of Supervised Machine Learning
 - I. Data: Preprocessing and Feature Extraction
 - II. Model:
 - I. Logistic Regression
 - III. Loss
 - IV. Optimization Algorithm
 - V. Inference

I. Data: Preprocessing and Feature Extraction

Features in Classification

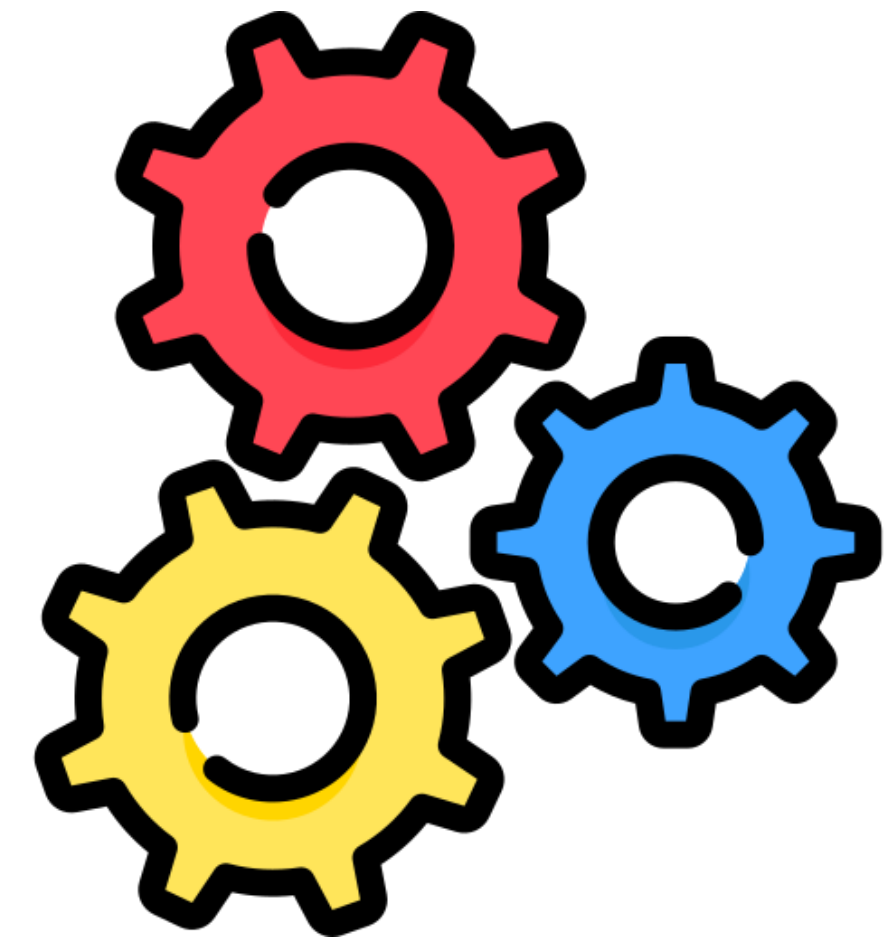
- Examples of feature x_i
 - x_i = "review contains 'awesome'"; $w_i = +10$
 - x_j = "review contains 'abysmal'"; $w_j = -10$
 - x_k = "review contains 'mediocre'"; $w_k = -2$
- Each x_i is associated with a weight w_i which determines how important x_i is
 - (For predicting the positive class)
- May be
 - manually configured or
 - automatically inferred, as in modern architectures



Can you guess the w for x_l = "review contains 'restaurant'"?

Data Pre-processing

- Documents containing raw texts must be preprocessed before feature extraction
 - Tokenization: splitting the text into units for processing
 - Removing extra spaces
 - Removing unhelpful tokens, e.g., external URL links
 - Removing unhelpful characters, e.g., non-alphabetical characters
 - Splitting punctuations: **Happy New Year!** -> **Happy New Year !**
 - Optional Steps
 - Contracting and standardizing: **won't** -> **will not**
 - Converting capital letters to lowercase: **New York** -> **new york**
 - Stemming or Lemmatization: **tokenization** -> **token**
 - In many of the above cases, pre-existing libraries exist for the task (e.g. Porter Stemmer)



Still relevant, especially for you to understand what current LLMs can automate!

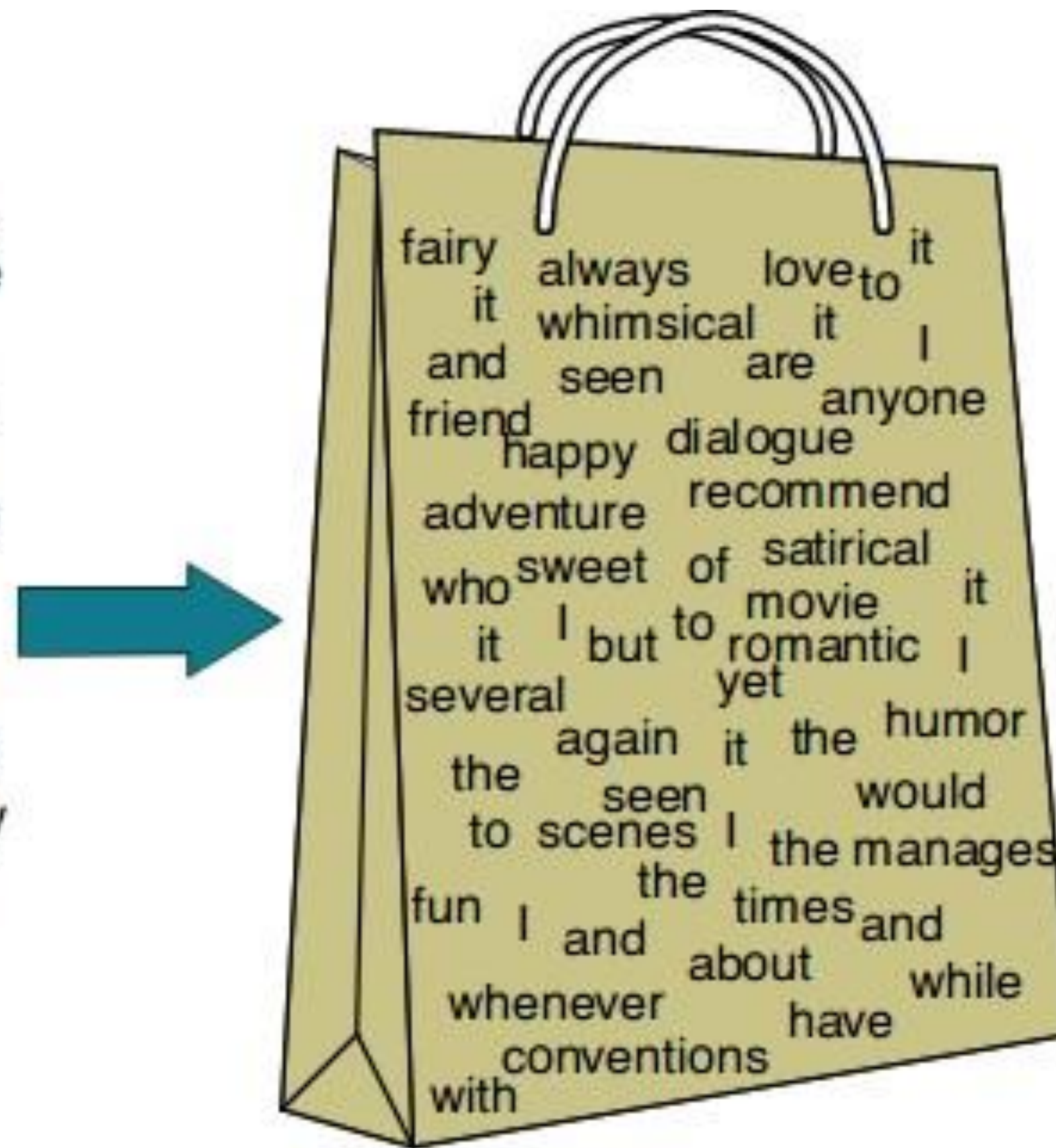
Feature Extraction

- Vocabulary Creation
 - A dictionary of all the words we care about
 - Excluding **stop words** from dictionary as they are useless for the task at hand
 - Mapping each word to a word id: **are** -> 2
 - Discarding words not included in the vocabulary
- Feature Representations:
 - Bag of Words (BoW)
 - Term Frequency / Inverse Document Frequency (TF-IDF)
 - Word Embeddings (e.g., word2vec, GloVe)

What happens when we see OOV words at test time?

Feature Representation: Bag of Words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Bag of Words

- With a word vocabulary of k words, BoW represents each doc / review x into a vector of integers
- $\mathbf{x} = [x_1, \dots, x_k]$, $x_i \in 0, 1, 2, \dots$
 - $x_i = j$ indicates that word i appears j times in the doc / review

Vocab = [good, bad, nice, expensive, love]

“I love this shirt because it is nice and warm. The fabric is also nice and the color complements my skin tone.”

→ [0, 0, 2, 0, 1]

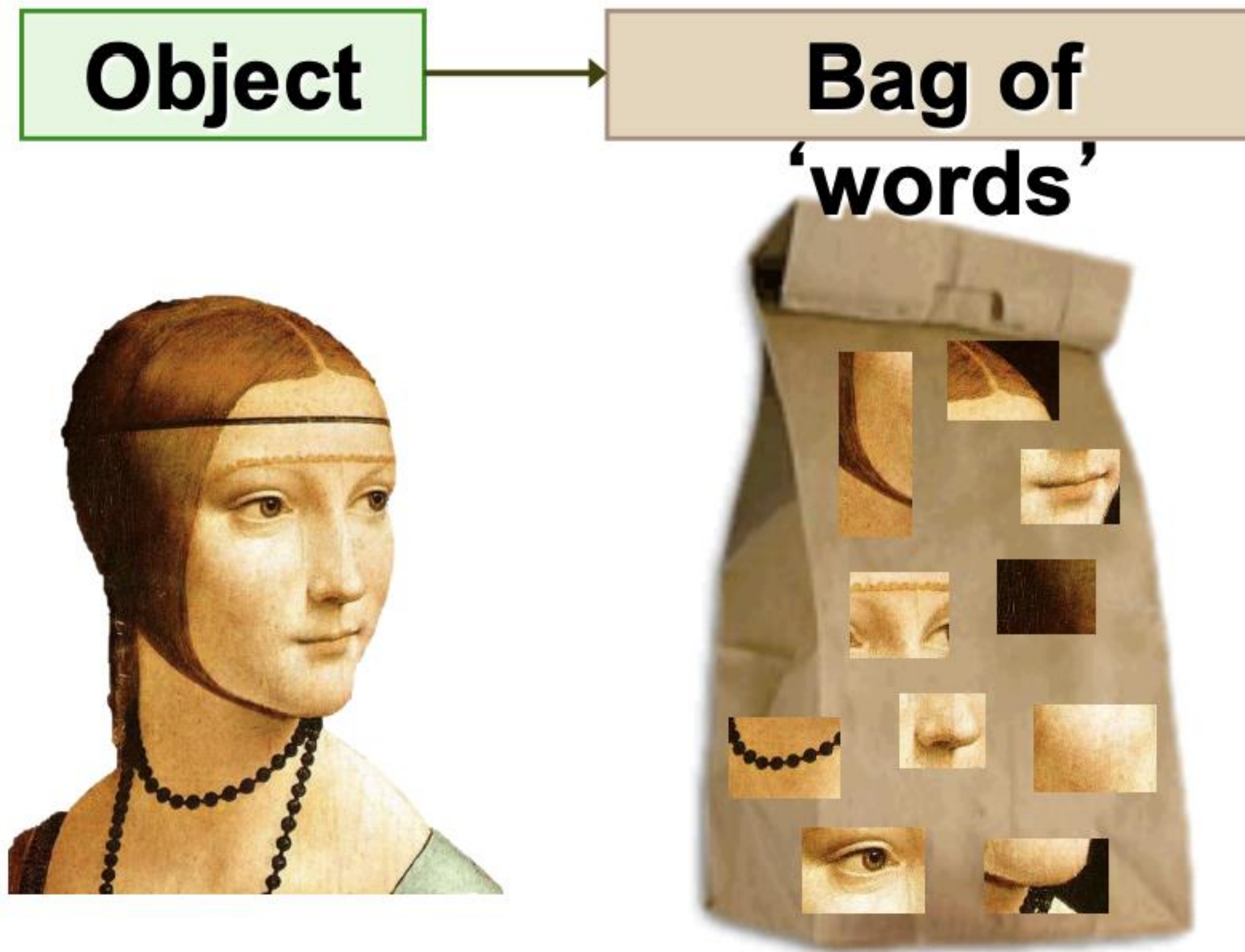
Bag of Words: Pros and Cons



- Limitations:
 - Insensitive to language structure: all **contextual information** has been discarded
 - Information in word dependencies is overlooked: **new york** vs **new book**
 - The resulting vectors are just word counts and are highly sparse
 - Dominated by **common words**
- Pros:
 - Simple!
 - Leads to acceptable performance in quite a few settings

Solutions?

Bag of Visual Words



Next Class:
II. Model:
(a) Logistic Regression